

Escuela de Estadística

XS-0129 Programación para Estadística I

R: GIT



Agenda

- GIT y su importancia
- Conceptos básicos
- Flujo de trabajo de GIT
- Principales acciones en GIT
- Instrucciones en GIT
- Github
- Práctica

Git y su importancia

Git es un **sistema de control de versiones** distribuido, diseñado para manejar proyectos de todo tamaño con rapidez y eficiencia. Fue creado por Linus Torvalds en 2005, principalmente para el desarrollo del kernel de Linux. Aquí hay algunas características clave de Git:

- **Control de versiones:** Git permite a los desarrolladores llevar un registro de los cambios en el código fuente a lo largo del tiempo. Esto facilita la colaboración, ya que múltiples desarrolladores pueden trabajar en el mismo proyecto simultáneamente sin sobrescribir el trabajo de los demás.
- **Distribuido:** A diferencia de los sistemas de control de versiones centralizados, en los que hay un solo repositorio central, Git permite que cada desarrollador tenga una copia completa del historial del proyecto en su propia máquina. Esto mejora la velocidad y la flexibilidad, permitiendo trabajar sin conexión y realizar operaciones locales rápidamente.

Git y su importancia

- ***Branching y merging***: Git facilita la creación y fusión de ramas (branches), lo que permite a los desarrolladores trabajar en características nuevas o corregir errores de manera aislada del resto del proyecto. Las ramas pueden fusionarse fácilmente, incorporando los cambios en la rama principal (main o master).
- **Integridad de datos**: Git usa una estructura de datos que garantiza la integridad del contenido. Cada archivo y cambio es identificado por un hash criptográfico único (SHA-1), asegurando que los datos no se corrompan ni se alteren sin ser detectados.

Git y su importancia

- **Colaboración:** Git permite a los desarrolladores compartir cambios mediante repositorios remotos, como los hospedados en plataformas como GitHub, GitLab, o Bitbucket. Esto facilita la colaboración en proyectos de código abierto y en equipos distribuidos.
- **Historial detallado:** Git guarda un historial detallado de todos los cambios realizados en el proyecto, incluyendo quién hizo el cambio, cuándo y por qué. Esto ayuda a rastrear errores, entender la evolución del proyecto y revertir cambios si es necesario.

Conceptos básicos de GIT

Los conceptos básicos de Git son fundamentales para entender cómo utilizar esta herramienta de control de versiones de manera efectiva. Aquí están los más importantes:

- **Repository** (Repositorio o “repo”):
 - Un repositorio en Git es un lugar donde se almacenan todas las versiones del proyecto, incluidos todos los archivos y el historial de cambios. Puede ser local (en su máquina) o remoto (en servidores como GitHub, GitLab, Bitbucket, etc.).
- **Commit:**
 - Un *commit* es un registro de los cambios hechos en el repositorio. Cada commit tiene un identificador único (hash) y contiene un mensaje descriptivo sobre los cambios realizados. Es como tomar una “foto instantánea” del proyecto en un momento específico.

Conceptos básicos de GIT

- **Branch (Rama):**

- Una rama (branch) es una línea paralela de desarrollo. La rama principal suele llamarse *main* o *master*. Crear ramas permite trabajar en características nuevas o corregir errores de forma aislada, sin afectar la rama principal.

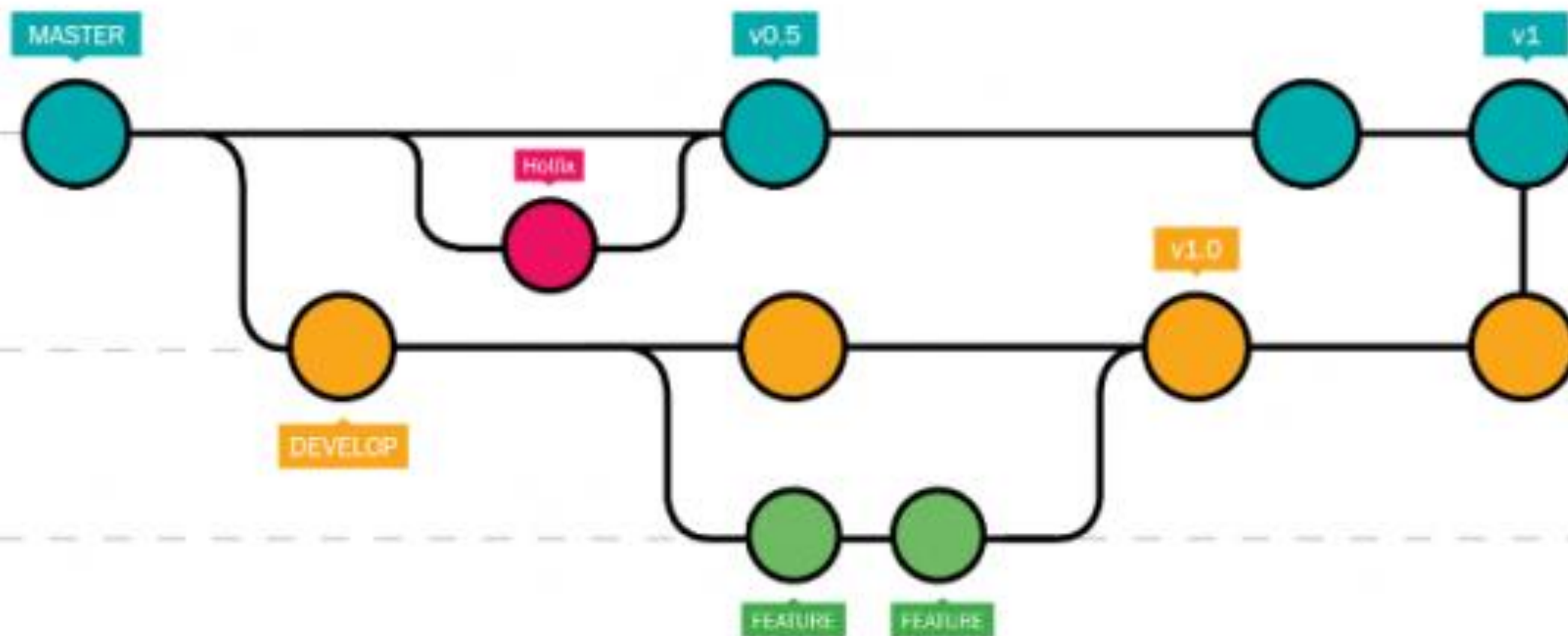
- **Instrucciones:**

- Son las instrucciones o comandos específicos con los cuales se versiona el código y se realiza todo el flujo de GIT.

- **Staging Area:**

- La "staging area" de Git, también conocida como "index" o "cache", es un espacio intermedio donde puede preparar los cambios antes de confirmar (commit) esos cambios en el repositorio. Es una característica clave de Git que permite un control más granular sobre lo que se incluye en un commit.

Flujo de trabajo en GIT



Principales acciones en GIT

- **Clone:** crear una copia exacta del repositorio remoto en su máquina local. Esto incluye todo el historial del proyecto y todas las ramas.
- **Fetch:** trae los cambios del repositorio remoto a su repositorio local, pero no los fusiona automáticamente con tu rama actual. Es útil para revisar los cambios antes de integrarlos.
- **Merge:** proceso de integrar cambios de una rama a otra. Esto es comúnmente utilizado para incorporar cambios de una rama de características o de corrección de errores a la rama principal
- **Pull:** actualizar su repositorio local con los cambios más recientes del repositorio remoto. Es una combinación de los comandos fetch (traer los cambios) y merge (fusionarlos con tu rama actual).
- **Commit:** registro de los cambios hechos en el repositorio. Cada commit tiene un identificador único (hash) y contiene un mensaje descriptivo sobre los cambios realizados.
- **Push:** enviar sus commits locales al repositorio remoto, compartiendo sus cambios con otros colaboradores.
- **Checkout:** cambiar de una rama a otra o para restaurar archivos del historial a tu directorio de trabajo

Instrucciones en GIT

- **Configuración inicial:**

- Configurar el nombre de usuario y el correo electrónico:

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu.email@example.com"
```

- **Inicialización y clonación:**

- Inicializar un repo:

```
git init
```

Instrucciones en GIT

- **Inicialización y clonación:**

- Clonar un repo existente:

```
git clone <url_del_repositorio>
```

- **Estados y revisión:**

- Ver estado actual del repo:

```
git status
```

Instrucciones en GIT

- **Estados y revisión:**

- Ver historial de commits:

```
git log
```

- Ver los cambios entre commits o entre el repo local y el remoto:

```
git diff
```

- **Gestión de archivos:**

- Añadir archivo o todos al área de preparación (staging area):

```
git add <archivo>
```

```
git add .
```

Instrucciones en GIT

- **Gestión de archivos:**

- Confirmar los cambios:

```
git commit -m "Mensaje del commit"
```

- Eliminar un archivo y registrarlo en el próximo commit:

```
git rm <archivo>
```

- Mover o renombrar un archivo:

```
git mv <archivo_viejo> <archivo_nuevo>
```

Instrucciones en GIT

- **Sincronización:**

- Enviar (push) cambios al repositorio remoto:

```
git push origin <rama>
```

- Actualizar (fetch) últimos cambios del repositorio remoto:

```
git pull
```

```
git fetch
```

Instrucciones en GIT

- **Ramificación y fusión:**

- Crear una nueva rama:

```
git branch <nombre_de_la_rama>
```

- Cambiar a una rama diferente:

```
git checkout <nombre_de_la_rama>
```

- Crear una nueva rama y cambiar a ella inmediatamente:

```
git checkout -b <nombre_de_la_nueva_rama>
```

- Fusionar una rama con la rama actual:

```
git merge <nombre_de_la_rama>
```


Instrucciones en GIT

- **Ramificación y fusión:**

- Crear una nueva rama:

```
git branch <nombre_de_la_rama>
```

- Cambiar a una rama diferente:

```
git checkout <nombre_de_la_rama>
```

- Crear una nueva rama y cambiar a ella inmediatamente:

```
git checkout -b <nombre_de_la_nueva_rama>
```

- Fusionar una rama con la rama actual:

```
git merge <nombre_de_la_rama>
```

Instrucciones en GIT

- **Revertir cambios:**

- Deshacer cambios en el área de trabajo:

```
git checkout -- <archivo>
```

- Revertir un commit específico (creando un nuevo commit que deshace los cambios):

```
git revert <id_del_commit>
```

- Restablecer el área de preparación y el directorio de trabajo al último commit:

```
git reset --hard
```

- Restablecer el área de preparación al último commit, pero mantener los cambios en el directorio de trabajo:

```
git reset --soft
```

Instrucciones en GIT

- **Gestión de etiquetas:**

- Crear una nueva etiqueta:

```
git tag <nombre_de_la_etiqueta>
```

- Ver todas las etiquetas:

```
git tag
```

- Eliminar una etiqueta:

```
git tag -d <nombre_de_la_etiqueta>
```

Instrucciones en GIT

- **Repositorios remotos:**

- Agregar un repositorio remoto:

```
git remote add <nombre> <url_del_repositorio>
```

- Ver los repositorios remotos:

```
git remote -v
```

- Eliminar un repositorio remoto:

```
git remote remove <nombre>
```

Instrucciones en GIT

- **Otras utilidades:**

- Ver la configuración de Git:

```
git config --list
```

- Limpiar archivos no rastreados:

```
git clean -f
```

GITHUB

- GitHub es una plataforma de alojamiento para el control de versiones y la colaboración en el desarrollo de software. Utiliza Git para gestionar y controlar el código fuente de proyectos.
- Algunas características principales son:
 - Repositorio remoto.
 - Colaboración.
 - Integraciones y automatización (CI/CD).
 - Documentación y Wikis.
 - Gestión de proyectos.
 - Amplia comunidad.



Uso de GITHUB

1. Entrar a www.github.com
2. Crear una cuenta o accederla.